Module 26-1

Introduction to Assertion-Based Verification (ABV)

cadence

This page does not contain notes.

Module Objectives

In this module, you

- Gain an overview of Assertion-Based Verification (ABV)
- Discuss who writes assertions and for what purpose
- Describe the benefits and issues of using assertions

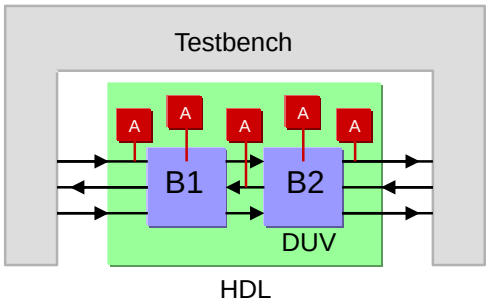
cadence

This page does not contain notes.

What Is an Assertion?

An assertion is a directive to an EDA tool to verify that a property is always true.

- A property is a description of design behavior.
- We instruct the tool what to do with the property using verification directives:
  - `assert, cover, assume, restrict`
- An assertion is a check that a property is always true.
- During verification, assertions continually observe:
  - If a specific condition occurs, or
  - If a specific sequence of events occurs.
- Assertions offer a way to significantly enhance productivity for designers:
  - Find bugs earlier and more easily.



3

cadence

An assertion is a directive to an EDA tool to verify that a property is always true (the terminology is that it “holds”). The simulator will report any failure of that design property to hold. Stating the assertion is the easy part. Crafting useful design properties is more difficult. However, it is far easier than debugging a broken system without the help of assertions.

The graphic illustrates where you can place an assertion. Note that you can place them on design inputs and design outputs and design interconnect and in the design blocks, as well as in a testbench.

The verification directives assume and restrict are used only for formal verification.

Cover and assert can be used in simulation or formal verification.

Why Use Assertions?

Improves observability

- Automatically and constantly checks behavior
- Can detect hard-to-find bugs embedded deep in the design
- Isolates problem close to source
- Can be applied non-intrusively to legacy designs

Improves verification efficiency

- Can concisely and unambiguously document design intent
- Encapsulation and reuse
- Can detect bugs earlier in the design cycle
- Trap and exit on errors, saving simulation cycles
- Writing assertions helps designers to better understand their design and do more debugging

Assertions can be embedded in code, to “travel with” design from:

- Project to project – ideal for IP
- Tool to tool – ideal for design methodology integrity

cadence

You can place assertions anywhere in your design hierarchy, enabling you to monitor design behavior locally and highlight problems as soon as they occur at the source of the problem. Conventional testbenches need to propagate a problem to the design outputs to detect it, and then you have to trace the error back through the design, and probably back through time as well, to determine the source. Assertions have the potential to detect problems during the early stages of testbench development, when the testbench might not yet be sufficiently robust to even propagate a problem to the outputs.

You can have the simulation terminate upon failure of critical assertions, thus permitting other projects to use the compute cycles that would otherwise be wasted.

Writing design properties encourages you to think more clearly about your design. This itself can highlight problems before you even finish the property. Overall verification efficiency is improved by designers producing better designs to begin with.

Your embedded assertions travel with your design as it is used in different projects, warning against unexpected or incorrect use of your design block.

Your embedded assertions also travel with your design from tool to tool, for example, from simulation to formal verification, avoiding duplication of the verification effort.

Who Writes Assertions and Why?

Who	Type of Assertion	Reason	Example
System Architects	Abstract properties	Ensure implementation meets functional spec.	An asynchronous FIFO does not indicate full and empty at the same time.
RTL Designer	Functional Property	Implementation has properties the RTL designer intended it to have.	When Read and Write pointers both 0, the FIFO indicates empty.
Verification Engineer	Functional Coverage	Ensure test stimulus exercises the corner cases as required by the test plan.	Did you read from full FIFO, write to the last empty FIFO location, can you go from empty to full then back to empty, etc.
IP Designer	Correct use of IP	Indicate to IP User that the IP is not being used correctly.	FIFO was read when it was empty.

A system architect develops abstract properties that represent the desired behavior of the system at the functional level. At this point, the actual implementation is not known.

The block designer develops objective properties that represent their intended behaviour of the system at the RTL level.

As an IP designer, the RTL designer will want to embed input assertions in the design to detect future misuse of the design.

The verification engineer focuses on design functional coverage points, which should include that the testbench actually exercised the asserted properties. Thus, functional coverage is a fundamental part of an Assertion-Based Verification methodology.

Issues with Assertions

- Assertions must be constructed with care.
  - Incorrectly specified assertions can give misleading results.
  - Debugging an assertion can be difficult.
- How do you know when enough assertions have been written?
- It is easy to shoot yourself in the foot using overly complex SVA constructs.
  - The syntax is “sugared”
    - The simplicity of the syntax hides complex overlapping behaviors.
- Quality of test stimulus is critical.
  - The simulator can make only those checks which the test exercises.
  - Coverage metrics reveal which checks are exercised.
- Checking assertions consumes CPU cycles.
- Other issues...

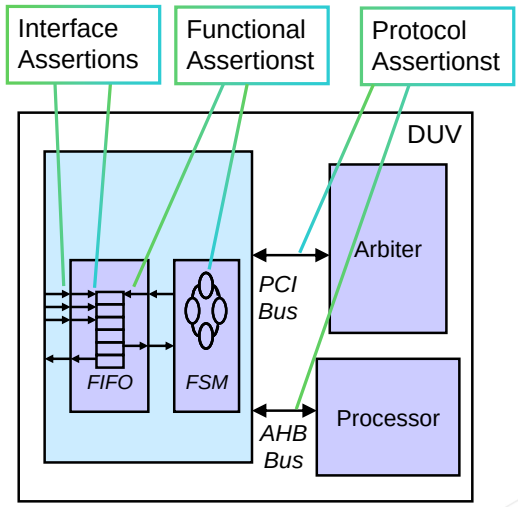
The most prominent issue with the Assertion-Based Verification methodology is that you, the user, must craft the design properties. It is easy to miss nuances of design behavior, some corner cases, that really ought to be checked. There is also the issue of “how do you know when your property set is sufficient”.

The simulator can check only those properties that the testbench exercises, so testbench quality is critical. This demands coverage metrics, which help to solve the “how do you know when you have tested enough” question, but do nothing for the “how do you know when your property set is sufficient” question.

What Is Assertion-Based Verification (ABV)?

Assertion-**B**ased **V**erification is the structured use of assertions to describe and verify design properties.

- Assertions monitor and report:
  - Expected behavior
  - Forbidden behavior
- Assertions are used by:
  - Static verification tools
    - No test vectors, formal (mathematical) proof
  - Dynamic verification tools
    - Dependent upon simulation test effectiveness (as measured using functional coverage)
- ABV also encompasses SVA constructs for defining functional coverage.



7

cadence

Assertion-based verification is a verification methodology that utilizes assertions. Assertions are not limited to just simulation tools. Formal verification tools, that do their verification statically, typically with very little helper test stimulus, if any, also use assertions. Coverage is an essential part of any verification activity whether one is using a simulator or a formal tool. SVA includes constructs for defining functional coverage.

ABV implies that we have a verification plan which details which properties are required to verify the design and which technologies the properties should be used with, for example, simulation, formal.

HDL-Based Assertions (Assertions Are Not New…)

You have been “asserting” correct design conditions all along, using:

```
if ( !good_condition ) $display ( error_message ) ;
```

```
`ifndef SYNTHESIS
always @GNT
  @(negedge clk)
  if ( GNT != 4'h0 )
    begin: GNT_CHK
      integer cnt, idx;
      cnt = 0 ;
      for ( idx = 0; idx <= 3; idx = idx+1 )
        if ( GNT[idx] != 1'b0 )
          cnt = cnt + 1;
      if ( cnt > 1 )
        $display ( "ERROR: Multiple GNT" );
    end
`endif
```


Verification personnel writing Verilog testbenches have been using an informal type of assertion right along. They write a process, that upon appropriate events, checks that some expression has the desired value, and if not, takes appropriate action. They can even write a check that spans multiple cycles, although that is more difficult, and thus rare.

Why Is ABV Better Than HDL-Based Assertions?

Assertion-Based Verification goes *well beyond* HDL-based assertions:

- Standard language constructs to concisely express complex temporal behaviors
  - PSL and SVA
- Standard tool support taking advantage of the new constructs
  - Failure messaging
  - Statistics gathering
  - Debug features
- Definition of Functional Coverage

The use of assertions is not new. What is relatively new is the standardization of language constructs with which you can concisely express temporal design behaviors, and consistent support of those constructs, with failure messages, statistics collection and reporting, and user-friendly debug features.

Module Summary

Complex properties are difficult to write using Verilog.

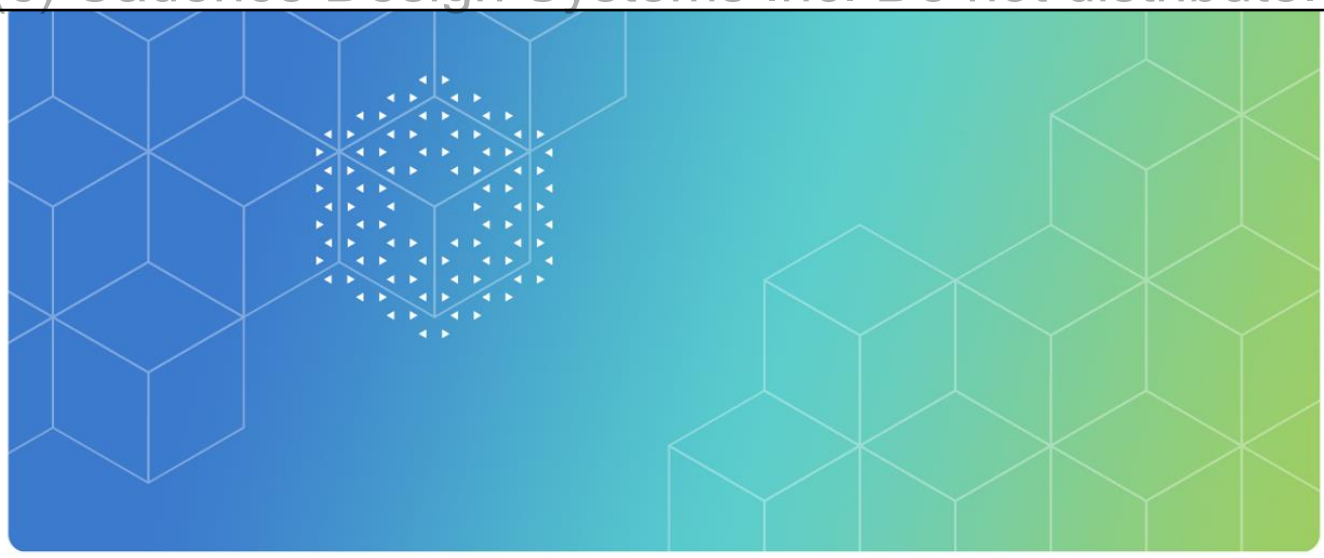
- A dedicated syntax like SystemVerilog Assertions helps greatly

Assertions can reduce debugging time by identifying incorrect design behavior when and where it occurs. The Assertion-Based Verification methodology does the following:

- Captures specifications
- Captures design assumptions
- Documents interfaces
- Provides white-box visibility in simulation
- Checks properties of legacy designs without modifying existing code
- Supports advanced verification technologies:
  - Simulation, acceleration, emulation
  - Functional coverage
  - Static (formal) verification

cadence

This page does not contain notes.



Module 26-2

Introduction to SystemVerilog Assertions

cadence

This page does not contain notes.

Module Objectives

In this module, you

- Examine the structure of a concurrent assertion
- Create a simple instantaneous concurrent assertion
- Use implication to create conditional assertions
- Define sequences to create multicycle assertions
- Use simple cycle and sequence repetition to create more efficient assertions

cadence

This page does not contain notes.

What Are Concurrent Assertions?

Concurrent assertions describe behavior that spans over time.

Unlike immediate assertions, the evaluation model is based on a clock so that a concurrent assertion is evaluated only at the occurrence of a clock tick.

- Concurrent assertions can be embedded in functional code and ignored by synthesis.
- They are supported by mainstream simulators and formal verification tools.
- You define and assert the functional properties of a design with Concurrent Assertions:
  - Boolean expression about some design behavior.
  - What it should or should not do.
  - Can be single or multiple cycle.
  - Uses Verilog syntax, yet …
  - Mathematically precise and computationally efficient.
    - Sufficiently expressive to specify “real world” design properties.

```
RW_CHK : assert property ( @(negedge clock)  !(wr_en && rd_en) );
```

13

cadence

We have learned that an immediate assertion is a procedural statement that some Boolean condition is true. The assertion is checked when the procedural statement executes.

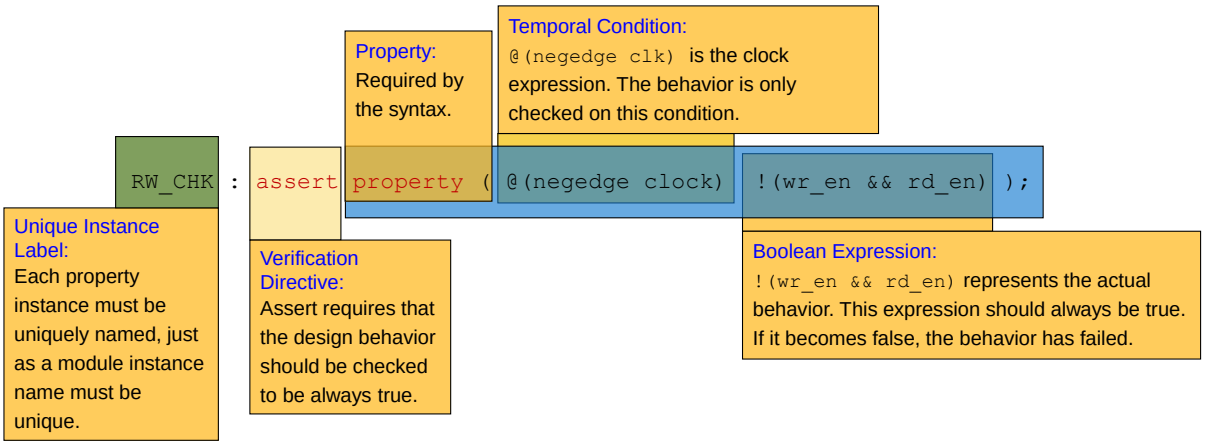
You typically place concurrent assertions outside procedural blocks to execute concurrently with the procedural blocks and utilize their own clock event. In simulation, this means each assertion runs continually in parallel and in the background.

A property is a design behavior that may span multiple cycles. It is a design requirement, similar to a synthesis constraint that the design must work with a 250MHz clock. SystemVerilog has special operators you use to develop concise property expressions to represent complex multicycle design behavior.

Concurrent Assertion Structure

Let us examine the following required design behavior.

"`wr_en` and `rd_en` are never both high at the negative edge of the clock"



This is one form of a concurrent assertions. There are other forms as you will see.

- The assertion starts with an optional statement label and the keyword `assert`. As designs typically contain hundreds if not thousands of assertions, a user-defined label will help with assertion management and debugging. The `assert` keyword tells the simulator to check this property.
- The `property` keyword defines this is a concurrent as opposed to an immediate assertion. The two terms of the property must be enclosed in parentheses.
- The first part of the property is the temporal condition which defines the event expression upon which the assertion is evaluated. All properties must be clocked, although the temporal condition can be defined elsewhere.
- The second part of the property is the Boolean expression, defining the design behavior which will be checked at the temporal expression. If the Boolean expression evaluates to 1, this simple assertion passes. If the expression evaluates to any other value (0 , X, Z), the assertion fails and an error message is reported.

The assertion label must be unique in the current scope. It is a good practice to use an uppercase label to avoid the likelihood of clashes with any other local identifiers. If you fail to label your assertions, the simulator just reports the scope of the assertion if it fails.

Concurrent Assertions Placement

- Properties are declared as normal SystemVerilog declarations:
  - Usually in a module or interface.
- Properties are asserted as a concurrent statement:
  - Usually in a module or interface.
  - Also legal inside an always or initial block.
    - Strong recommendation not to do this.

```
module mod1
  (input logic clk,
   ...
   output logic en1, en2);

always @(negedge clk)
   begin
     ...
     (en1,en2) <= sel[1:0];
     ...
   end

  property RW_CHK_CLK;
    @(negedge clk) en1 || en2;
  endproperty

  A1: assert property (RW_CHK_CLK);

endmodule
```


cadence

You can declare a property, like a normal SystemVerilog type declaration, in a compilation unit scope, package, interface, module or program.

The property is asserted as a procedural statement, only in an interface, module, or program. You can also assert a property in an always or initial block, although there are many issues with doing this, which are covered in the full SystemVerilog Assertions class.

If your tool flow contains tools which cannot compile SystemVerilog, then there are two common options:

- Declare your own text macro and use conditional compilation (`` `ifndef ``) to conceal the code from the non-compliant tool.
- Use the SystemVerilog `bind` construct. This allows you to place assertions within a module, and then instantiate the assertion module into an existing design module from a higher hierarchical scope, usually the testbench, without modifying the design module. Bind is covered in the full SystemVerilog Assertions class.

Concurrent assertions can also be placed inside of procedural blocks.

However, this makes there behavior very difficult to understand.

It is strongly recommended that you don't place concurrent assertions inside of an `always` or `initial` block.

Defining a Simple Property

IEEE 1800-2012 16.5

- Define a design behavior as a Verilog Boolean expression.
- Any expression allowed in an `if` condition can be used, including:
  - Functions
  - Out Of Module references
- Subject to normal Verilog restrictions, particularly:
  - Case sensitivity
  - Naming rules

```
en1 || en2;
```

```
((a + b) >= 42) && (c || d);
```

```
(num_set(sel_vec) == 1) && valid;
```

`num_set` is a user-defined functionext

cadence

A property expression can be simply a Boolean expression. Use parentheses where needed to appropriately group the operands and to enhance readability.

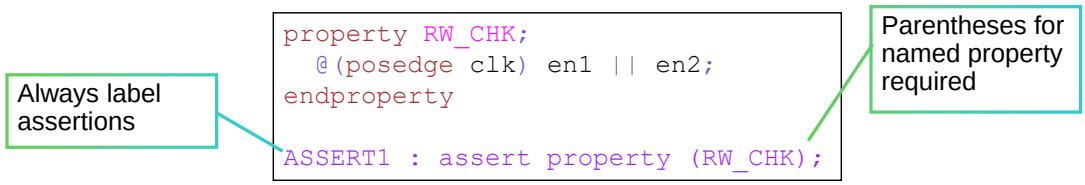
Naming and Asserting the Property

You have two options:

- Declare and instance the property on the same line.

```
ASSERT1 : assert property ( @(posedge clk) en1 || en2 );
```

- Declare and instance the property in separate statements:
  - Gives more flexibility
  - Property can have arguments



Instance labels are used to hierarchically reference the assertion.

17

cadence

Assertions can be defined in 1 of 2 ways:

- Single statement assertion – Define the Boolean condition as an un-named property and assert the property directly.
- Named property declaration – Declare a property to define the desired behaviour and assert the property in a separate statement. The property must have a name, and the assertion statement can be labelled.

The assertion label and property name can be used by the verification tools to identify the assertion. The name and label must be legal Verilog names that do not conflict with other identifiers in the same scope. Using uppercase identifiers for both label and name is a common convention to avoid name clashes.

The named property declaration form is more flexible for complex assertions. By separately declaring and naming properties, you can:

- Use the properties as building blocks in more complex assertions.
- Reuse properties for assertions, functional coverage and formal verification.
- Declare property arguments, allowing a single design behavior to be applied to different signals.

We will use the named property form throughout the course and assume the property is separately asserted if not explicitly shown.

Clocking the Property

- Properties are evaluated upon standard Verilog clocking events.
  - `@identifier` or `@(event_expression)`
  - Can also use SystemVerilog enhancements (such as `iff`)
- All assertions must be clocked!
- Clocking expression does not need to be:
  - A design hardware clock
  - Periodic
- Good clocking expressions can greatly simplify properties.
- A default clock can be specified.

```
property RW_CHK_CLK;
  @(negedge clk iff VALID) en1 || en2;
endproperty
```

```
property P3;
  @(negedge addr_en) addr <= 7;
endproperty
```

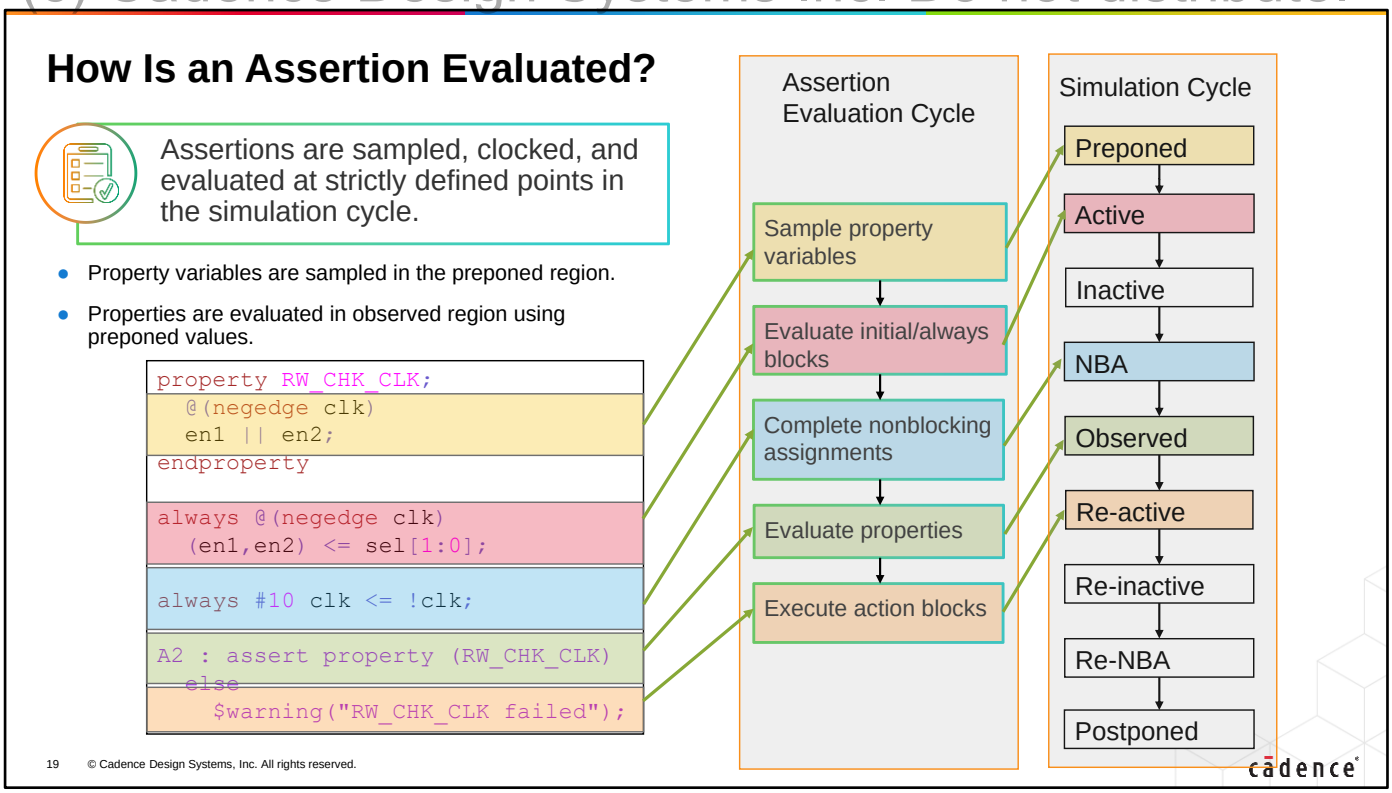

All assertions must be clocked. You can specify the clock in a sequence expression that the property uses or in the property specification.

The clocking event is typically just a Verilog event expression, exactly the same as that used in an RTL `always` procedure, or in embedded event control in verification code. SystemVerilog event expression enhancements, such as `iff` (if-and-only-if) qualifiers, can also be used.

Default clocks can be defined for properties which do not have an explicit clock in their declaration.

This is done with a clocking block and the keyword pair “default clocking”.

The default clock applies only to the scope in which it is declared.



A SystemVerilog simulation time instant is divided into several regions. The regions separate various activities to help avoid races between the writing and reading of a variable.

Assertions are sampled, clocked, and evaluated at strictly defined points in the simulation cycle. Variables in a property are sampled at the beginning of the simulation cycle in the *preponed* region. In the Active, Inactive and NBA regions, procedures are evaluated and variables updated. It can take several passes through these regions until you reach a steady state at this point in simulation time.

Then properties are evaluated in the *observed* region. If the temporal condition is currently true, such as a rising edge of `clk` occurred in the NBA region, then the assertion is triggered. However, the property is evaluated using the sampled variable values from the *preponed* region, which might be different from the current value of these variables in the *observed* region, that is, they were updated in the Active or NBA region.

This may seem counter-intuitive, but if you consider gate-level behavior, where the input sampled by a register is that in the setup time before the clock edge, then it makes more sense.

In the example above, `en1` and `en2` are updated on the `negedge` of `clk`, and the property `RW_CHK_CLK` is evaluated on the `negedge` of `clk`. `en1` and `en2` are sampled in the *preponed* region. `clk`, `en1` and `en2` are updated in the NBA region, and if `clk` fell, the property is evaluated in the *observed* region, using the sampled values of `en1` and `en2`. Therefore, assertion evaluation uses the preclock values, and the values of `en1` and `en2` in the *observed* region (post-`clk` values) might be different from the values used in the property evaluation (pre-`clk` values).

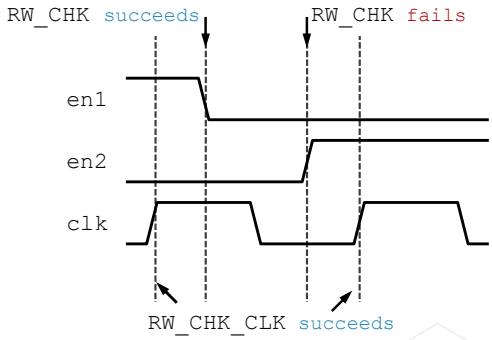
This diagram of the time slots is greatly simplified and omits all the regions used for PLI callbacks.

Example: Assertion Evaluation

- `RW_CHK` evaluated when `en1` changes or `en2` changes.
  - 2 evaluations, 1 failure
    - Using preponed values before change.
- `RW_CHK_CLK` evaluated on the positive edge of `clk`.
  - 2 evaluations, 2 successes
    - Using preponed values before rising edge `clk`.
- There are many things to understand to avoid problems if signals used as clocking signals are also referred to in the property behavior.
- Strongly recommend not to create such properties, for example, `RW_CHK`.

```
property RW_CHK;
  @(en1 or en2) en1 || en2;
endproperty

property RW_CHK_CLK;
  @(posedge clk) en1 || en2;
endproperty
```



Clock asynchronous properties on any change of any signal used in the property expression.

Clock synchronous properties on the appropriate edge of the appropriate clock signal.

Property expression evaluation uses the sampled values, that is, the values before the clock event.

A side effect of using sampled values in assertions rather than instantaneous values is that conflicting behavior is not immediately detected. Notice that the failure of `RW_CHK` occurs some time after the incorrect behavior started. Indeed if we end the simulation before `en2` changes from 0->1, then we would never be aware that there was a problem.

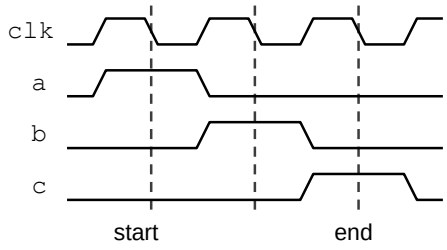
What Is a Sequence Expression?

A **Sequence Expression** describes a series of one or more cycles of design signal states (each cycle described by a Boolean expression).

- Simple Boolean properties are instantaneous:
  - Single pass/fail test at the evaluation point
- Multi-cycle properties are described using sequences:
  - Series of Boolean equations
  - Evaluated in successive clocking cycles
  - Each cycle separated by `##N`
- Sequences are building blocks of properties.
- There is a wide range of operators and features to construct complex sequences:
  - *if-then* implication
  - Repetition
  - Composition operators

Sequence

```
@(negedge clk)
  a ##1 b ##1 c;
```



21 © Cadence Design Systems, Inc. All rights reserved.

cadence

Simple Boolean properties are instantaneous – they will pass or fail at a single evaluation point. However, most properties are temporal. They require different checks to be performed over different cycles to confirm the design behavior.

A sequence is a series of Boolean conditions over successive cycles. You use `##` (cycle delay operator) to separate one evaluation cycle from the next. The example sequence:

```
a ## b ## c
```

is evaluated as condition `a` true in the first cycle, followed by condition `b` true on the next, followed by condition `c` true on the next cycle. If each cycle of a sequence is true, then the whole sequence is true. The first condition which is not true causes the sequence to fail on that cycle.

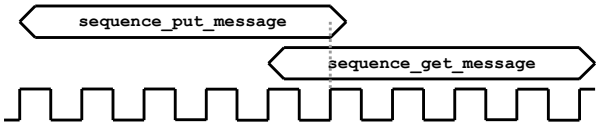
Sequences are used as building blocks of properties and there are a wide range of operators and constructs to define conditional, repeated, overlapping, parallel, alternate and nested sequences to help create properties.

Defining a Cycle Delay

IEEE 1800-2012 16.7

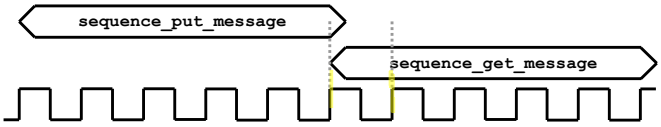
Delay of 0 cycles, AKA **Sequence Fusion**

```
sequence_put_message ##0 sequence_get_message
```



Delay of 1 cycle, AKA **Sequence Concatenation**

```
sequence_put_message ##1 sequence_get_message
```



cadence

You can delay sequence evaluation by a constant number of cycles. The delay can be any integral number between zero and the end of simulation.

A delay of 0 cycles, that is, sequence fusion, places the first cycle of the following sequence coincident with the last cycle of the preceding sequence.

A delay of 1 cycle, that is, sequence concatenation, places the first cycle of the following sequence after the last cycle of the preceding sequence.

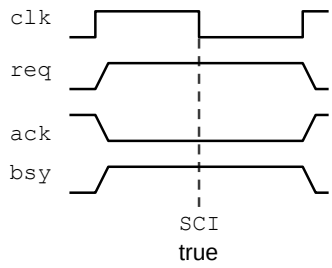
Implication (|-> : Same Cycle)

IEEE 1800-2012 16.12.6

- Conditional properties are defined with implication operators:
  - If `expra` is true, then `exprb` must occur.
  - If `expra` is not true, `exprb` is not checked.
- For the same cycle implication, `exprb` must be true in the same cycle as `expra`.
- For implication properties:
  - `expra` is the antecedent or enabling condition.
  - `exprb` is the consequent or fulfilling condition.
  - Both can be either sequences or Boolean conditions.

```
expra |-> exprb;
```

```
property SCI;
  @(negedge clk)
    (req && !ack) |-> bsy;
endproperty
```



23

cadence

Conditional properties are those where some initial (or enabling) conditions must be true before a series of (fulfilling) conditions must be checked. Conditional properties are defined using implication operators.

SystemVerilog provides two sequence implication operators. This is the “same cycle” implication operator. If the condition on the left of the operator is true, then the condition on the right-hand side of the operator must be true in the same cycle.

The left side is called the antecedent or enabling condition.

The right side is called the consequent or fulfilling condition.

In this example, if `req` is high and `ack` is low at the failing edge of `clk`, then `bsy` must be high in the same cycle. If `req` is low or `ack` is high, then the enabling condition is not true, and `bsy` is not checked. As both sides of the operator are single Boolean conditions, the property can be rewritten as an unconditional property:

```
@(negedge clk) (req && !ack && bsy) | !req | ack;
```

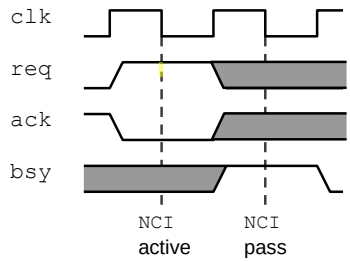
Either of the enabling or fulfilling conditions can be sequences. If an enabling sequence runs to completion, then the first cycle of the fulfilling sequence (or condition) is checked in the same cycle as the enabling sequence completes.

Implication (|=> : Next Cycle)

- For the next cycle implication, `exprb` must be true in the cycle after `expra` completes.
- Otherwise, same rules apply:
  - If `expra` is true, then `exprb` must occur.
  - If `expra` is not true, `exprb` is not checked.
  - Both can be either sequences or Boolean conditions.
- If a variable is not included in a condition, its value is “don’t care”:
  - `bsy` is not checked in enabling condition.
  - `req` and `ack` are not checked in the fulfilling condition.

```
expra |=> exprb;
```

```
property NCI;
  @(negedge clk)
    (req && !ack) |=> bsy;
endproperty
```



This is the “next cycle” implication operator. If the condition on the left of the operator is true, then the condition on the right-hand side of the operator must be true from the next evaluation cycle.

The left-hand side is called the antecedent or enabling condition.

The right-hand side is called the consequent or fulfilling condition.

At each evaluation point (falling edge of `clk` in these examples), the enabling condition is checked (`req` = 1; `ack` = 0). If the enabling condition is false, the assertion is ignored for this clock cycle. If the enabling condition is true, then the fulfilling condition is checked at the next evaluation point (`bsy` = 1 at next falling edge of `clk`). If the fulfilling condition is true, then the assertion passes. If the fulfilling condition is false, the assertion fails.

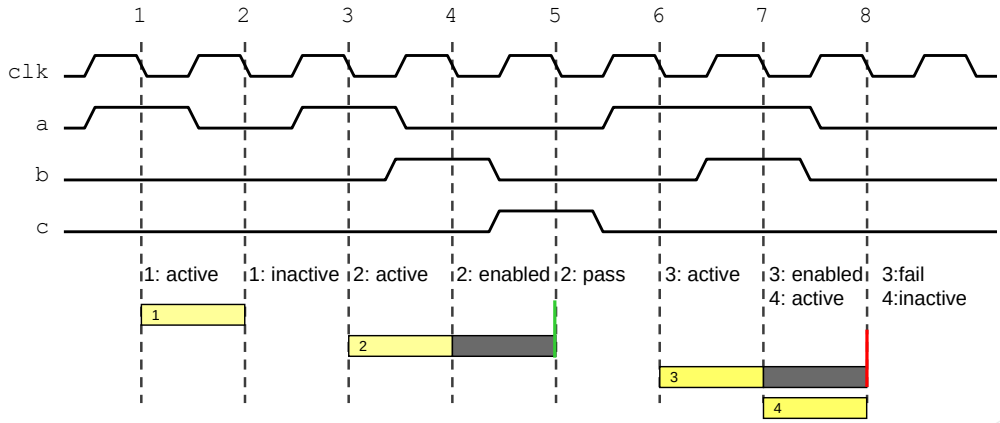
Either of the enabling or fulfilling conditions can be sequences. If an enabling sequence runs to completion, then the first cycle of the fulfilling sequence (or condition) is checked in the next cycle after the enabling sequence completes.

If a variable is not included in the enabling condition or fulfilling condition, then its value is “don't care”. You must be careful to write design properties which check all parts of the required behavior.

For example, should `bsy` be low in the enabling condition? Should `req` and `ack` stay unchanged in the fulfilling condition?

Analyzing Sequence Property with Implication

```
property STATE;
  @(negedge clk) (a ##1 b) |=> c;
endproperty
```



Remember that the enabling sequence on the left side of the implication operator must complete before the fulfilling sequence on the right side is checked.

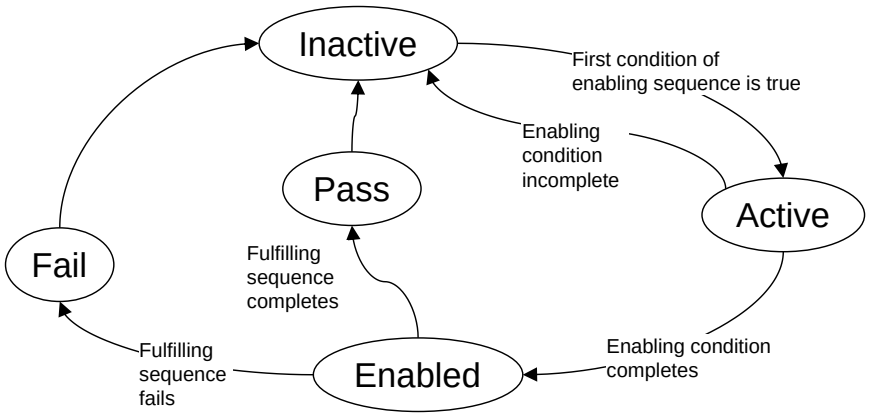
Here is an analysis of the evaluation property `STATE`:

- Cycle 1: `a` is true, so attempt 1 of `STATE` evaluation becomes active.
- Cycle 2: `b` is false, so attempt 1 of `STATE` evaluation becomes inactive.
- Cycle 3: `a` is true, so attempt 2 of `STATE` evaluation becomes active.
- Cycle 4: `b` is true, so attempt 2 of `STATE` evaluation becomes enabled.
- Cycle 5: `c` is true, so attempt 2 of `STATE` evaluation succeeds and finishes.
- Cycle 6: `a` is true, so attempt 3 of `STATE` evaluation becomes active.
- Cycle 7: `b` is true, so attempt 3 of `STATE` evaluation becomes enabled. `a` is still true, so attempt 4 of `STATE` evaluation becomes active.
- Cycle 8: `c` is false, so attempt 3 of `STATE` evaluation fails. `b` is false, and attempt 4 of `STATE` evaluation becomes inactive.

The key point here is that assertions can overlap. The first condition of the enabling sequence is checked at every evaluation point. If the first condition is true, another copy of the property becomes active, regardless of how many other copies of the property are currently active. There can be any number of copies of a property active at any one time, all at different stages in their evaluation.

Pictorial View of Assertion Property Evaluation States

An assertion can be in one of several states:




```
@(clocking) disable iff (dis_expr) enable |=> fulfill;
```


Assertions start in the *inactive* state. At each evaluation cycle, the enabling condition (or the first condition of an enabling sequence) is checked. If the condition is true, a copy of the assertion becomes *active*. In the *active* state, the assertion continues checking the enabling condition or sequence. If the enabling condition(s) do not complete, then the assertion becomes *inactive*. If the enabling condition(s) complete, then the assertion is *enabled*. In the *enabled* state, the assertion checks the fulfilling condition(s). If the fulfilling conditions fail to complete, then in the evaluation cycle the failure was detected, the state of the assertion is *fail*, after which it becomes *inactive*. If the fulfilling condition(s) are true, then the assertion state is *pass* in the evaluation cycle the final fulfilling condition becomes true.

A disable can terminate an *active* or an *enabled* assertion. For SystemVerilog2005, a disabled assertion counted as a pass. For formal verification, a disabled assertion is interpreted as a pass. From SystemVerilog2009, a disabled assertion has a new separate disabled state before returning to inactive.

Remember the enabling condition of every assertion is checked on every evaluation cycle, therefore multiple versions of a single assertion can exist in different states. This can make management and debugging of assertions difficult.

Disabling Properties with `disable iff`

IEEE 1800-2012 16.15

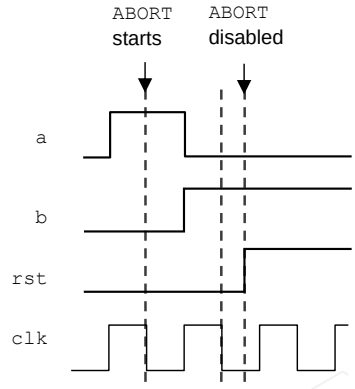
- To terminate an assertion when a condition is true, use `disable iff`.
- If `expr` is true **at any time**, then the property is terminated.
- Disable expression must be bracketed `()`.
- Useful for cancelling multi-cycle sequence properties.
  - For example, burst on a bus interrupted by reset.
- A default disable can be specified.

```
property NAME;
  @(clocking) disable iff (expr)
    the_property;
endproperty
```

Syntax

```
property ABORT;
  @(negedge clk) disable iff (rst)
    a |=> b ##1 c ##1 d;
endproperty
```

Example



27 © Cadence Design Systems, Inc. All rights reserved.

cadence

Disable is essential for terminating long sequence properties if design conditions change, e.g., in this case a reset occurs.

If the disable condition (`rst`) is true, then the assertion is disabled (terminated) immediately, regardless of how the property is clocked.

You can view the disable as being asynchronous from the property sampling clock.

Up to SystemVerilog2009, disabled properties were defined as a pass, just as if the property had successfully completed.. This could possibly lead to misleading results when a simulator reported the number of design properties which have passed or failed. This is why most simulators reported the number of assertions which have completed or finished, rather than the number which have passed.

From SystemVerilog2009, a new completion state for properties was added, in addition to pass or fail. When the disable expression of a property is true, the property is counted as “disabled”, and simulators can now report the number of assertions that are disabled.

In Formal Verification (where assertion languages were first used), disabled assertions are always counted as passes because the absence of a failure is deemed as a pass.

Default disable iff can be defined for properties which do not have an explicit disable iff in their declaration.

The syntax is

default disable <Boolean>, for example:

disable iff (!rst_n)

The default disable applies only to the scope in which it is declared.

Cycle Delay Repetition

IEEE 1800-2012 16.7

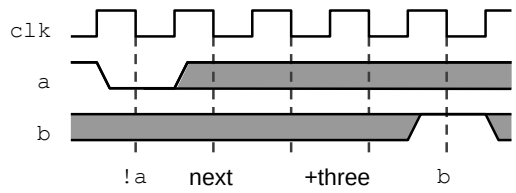
- `##` can be used with any constant integral expression to define a number of cycle delays.
- After `expr1`, count `N` evaluation points, after which `expr2` must be true.
- `##0` is allowed (and useful) for sequences:
  - Zero delay ("fusion")

```
expr1 ##N expr2
```

Use with care!

If `a` is low, then `b` is high `4` cycles later

```
property A3B_NEXT;
  @(negedge clk) !a |=> ##3 b;
endproperty
```



28

cadence

Cycle delay `##` just counts evaluation cycles without checking any conditions. The cycle delay expression can be an integral constant expression or a range between two integral constant expressions. Here we have a 3-cycle delay. The property succeeds if, after `a` is low, `b` is low 4 cycles later. Note that if the cycle delay comes immediately after the next cycle implication operator, then the cycle count starts after the cycle when the enabling condition completes. If same cycle implication is used, then the count starts on the same cycle as the enabling condition completes.

A delay of zero is valid and useful. A zero delay between the end of the left sequence and the start of the right sequence means that the left sequence ends and the right sequence begins in the same cycle. This is called sequence fusion.

Example

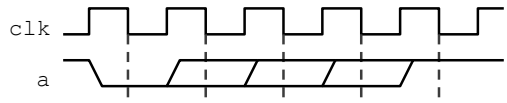
```
(a ##1 b) ##0 (c ##1 d) = (a ##1 (b && c) ##1 d)
```

Cycle delay repetition should be used with care. You are not checking any values during the cycle count, and you ask yourself whether you should be checking something. In the example above, should `a` be high or low during the 3-cycle delay? Can `b` only be low on the fourth cycle after `a`, or can it be low on every cycle?

Consecutive Sequence Repetition

IEEE 1800-2012 16.9.2

`a` is never low more than 4 cycles



You can specify multiple consecutive repetitions of a sequence using `[*N]`:

- Consecutive repetition operator.
- `expr` repeats consecutively `N` times.
- N must be an non-negative integer constant expression.

```
expr[*N]
```

Instead of this...

```
property SEQ_COUNT;
  @(negedge clk) !a ##1 !a ##1 !a ##1 !a |=> a;
endproperty
```

Write this...

```
property CONSEC;
  @(negedge clk) !a[*4] |=> a;
endproperty
```


cadence

SystemVerilog provides three sequence repetition operators. The consecutive repetition operator simply repeats an expression for a set number of times given by `N`. The consecutive repetition operator can be used with a Boolean expression, a named sequence, or an inline sequence enclosed within parentheses (see below).

Consecutive repetition allows an individual condition or a complete sequence to repeat, consecutively, a set number of times.

Example

`(a[*4])` is equivalent to `(a ##1 a ##1 a ##1 a)`

`((a ##1 b)[*2])` is equivalent to `(a ##1 b ##1 a ##1 b)`

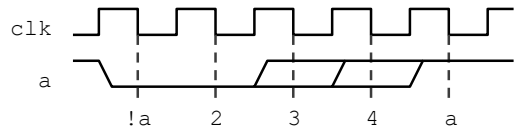
The count value `N` must be statically computable, that is, known at compilation time and not defined by a variable or expression.

Consecutive Repetition with Ranges

IEEE 1800-2012 16.9.2

`expr[*min:max]`

- You can specify a range for consecutive repetition.
- `expr` repeats consecutively for:
  - At least `min` number of times.
  - At most `max` number of times.
- Range limits must be non-negative integer constant expressions:
  - min can be `0`.
    - That is, no repetitions.
  - max can be **$** (unlimited).
    - That is, can repeat forever.
  - min is less than or equal to max.




```
property RANGE;
  @(negedge CLK)
    a ##1 !a |=> !a[*1:3] ##1 a;
endproperty
```

`$` bound can lead to ambiguities

`a` goes low in between 2 and 4 cycles only

The consecutive repetition operator can have a range. The range specifies a minimum number of repetitions which must occur and a maximum number which must not be exceeded.

Both bounds of the range must be statically computable (i.e., a constant expression) and must be defined in an ascending direction.

In the example, once the enabling condition of a falling edge on `a` is found, the property will check that `a` is false for two cycles (lowest bound of range). We use a same cycle implication operator to begin the count of `a` low in the same cycle as `a` low for the enabling condition completion. Then for the next 3 evaluation points, if `a` is true, the property completes and passes. Otherwise, the property remains active. On the final (fourth) cycle, `a` must be true (upper bound of range exceeded) or the property fails.

This property specifically looks for a falling edge on `a`, as we interpret the "`a` goes low" of the specification to imply this. The property would not catch the case of a starting low at the beginning of simulation, and then remaining low for more than 4 cycles.

A minimum bound of 0 means that the sequence or condition can be missing. For example:

```
a ##1 b*[0:2] ##1 c
```

matches

```
a ##1 c
a ##1 b ##1 c
a ##1 b ##1 b ##1 c
```

Additional SVA Features

SystemVerilog has many more advanced properties, sequence operators and features:

1. Conditional property operators
2. Nonconsecutive sequence repetition
3. Concurrent, alternate and overlapping sequences

Advanced SVA features:

- Detecting the end of a sequence
- Parameterized sequences and properties
- Action blocks on property pass or failure
- Functional coverage

The SystemVerilog Assertions class covers all the above including the following:

- Coding guidelines and recommendations
- Practical examples
- Formal Friendly SVA coding
- Use of Verilog helper code to simplify verification using SVA properties

cadence

SystemVerilog has many more advanced property and sequence operators and features. The *SystemVerilog Assertions* training covers these plus coding guidelines and recommendations, functional coverage and working examples.

Module Summary

- SystemVerilog assertions are based on Verilog Boolean conditions
- Constructing a simple Boolean assertion is straightforward:
  - Write a Boolean condition using Verilog syntax
  - Define when the condition is checked with a clock expression
  - Name the condition using the `property` statement
  - Assert the property using an `assert` statement
- With sequences you can easily express complex, multi-cycle properties:
  - Boolean expressions separated by cycle delay `##N`
- SVA has many features for defining multi-cycle and repeated sequences:
  - For example: consecutive repetition `[*N]`

Pause here for a moment and review what you have learned about SystemVerilog assertions.